

Deal or No Deal LangGraph 简化设计

SIMPLE 01

两个 AI 自动博弈 / 人类只观看

用你已学过的「模型 - 工具 - 条件边」循环，组织开箱、报价、还价与结算。

TypedDict / 增量更新

资料第 5 页

一个共享状态保存整局数据

AI Message / Tool Message

资料第 5-7 页

两个模型入口，共用一个执行节点

条件边 / compile / invoke

资料第 7-8 页

路由负责轮流行动与自动结束

01 STATE: 记住什么

GameState(TypedDict)

game: dict

箱值(私有)、自留箱、已开箱、轮次、阶段、报价、来电额度与还价标记。

actor: player / banker

pending: AIMessage | None

当前行动方；待裁判处理的模型候选。

player_messages

banker_messages

两份消息列表独立，均使用

Annotated[..., add_messages]

retry_count / model_calls

status / result

status = running / completed / error

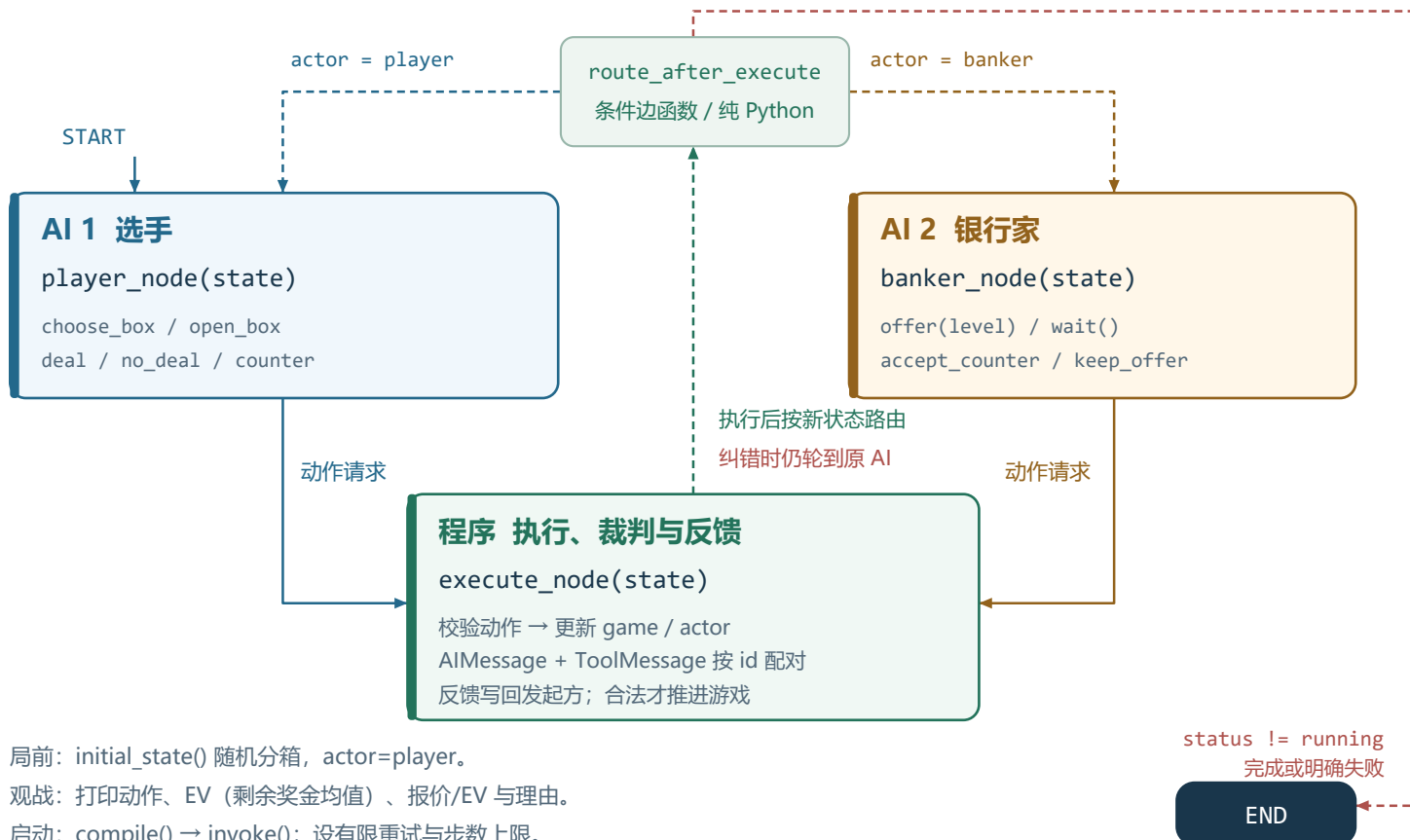
普通字段覆盖；节点返回更新字典。

API 输入：角色提示 + 本方历史 + 公开局面。

可共用同一模型；隐藏箱值映射不外发。

02 GRAPH: 三个节点 + 一个路由函数

路由框不是第四个节点



03 银行家提前来电仍保留

开箱后先判轮未；否则检查提前来电资格。
最多 2 次/局、1 次/轮；拒绝后继续原轮。

04 工具协议与结束条件

每次一个动作；错则反馈原 AI，最多重试 2 次。
无工具请求不等于结束；仅程序能确认终局。

05 当前进度就能搭起的版本

用状态、普通函数、消息配对与条件边完成。
仅内存运行；不接人类中断、检查点或并行。